

CWC 2.0 ESM — Service Management System Handover Guide (Maintainer README)

Audience: the next engineer taking over the **Enterprise Service Management (ESM)** module — the IT / HR / Finance service desks and the core request platform.

Purpose: a living codebase guide so you can find your way, understand the architecture, and work safely.

Scope: the **service-desk / request platform** only — the IT Support, HR Services, and Group Finance desks, the request lifecycle, the workflow-designer engine, approvals, SLA/escalation, notifications, IT asset management, and knowledge base. The **Credit LOS** and **CRM** modules are separate (see note below).

> **Three "modules" in this monorepo:** (1) the ESM service-management platform (this doc), (2) the Credit LOS module (backend/src/credit/, frontend/pages/credit/, docs in docs/handover/), and (3) CRM (backend/src/routes/crm.routes.ts, frontend/pages/Crm*.tsx). This handover is for **ESM only**; its docs live in docs/handover-esm/.

1. What this module is

The ESM module is the **service desk / request management core** of CWC 2.0. Employees submit requests through three service desks, requests flow through a **workflow engine** (published workflow graphs), are approved/escalated per SLA, and close out with asset/KB side effects.

```
Employee submits (IT / HR / Finance / ESM-Travel)
→ Request created (with form data snapshot)
→ Workflow engine drives status transitions (DB-first validation)
→ Approvals (entity/policy/delegation) + SLA/escalation timers
→ Notifications (in-app SSE + email, durable outbox)
→ Resolution / closure (asset provisioning, KB, reports)
```

Scale (verified)

- Core route files: ~46 in backend/src/routes/ (request, serviceDesk, workflow, approval, sla, asset, kb, notifications, onboarding/offboarding/loan/interview/screening, it/finance/chargeback/esm-workflow, etc.)
- Core controllers: ~45 in backend/src/controllers/
- Core services: ~70 in backend/src/services/
- Frontend: frontend/pages/*.tsx (Dashboard, ITSupport, HRServices, GroupFinance, MyRequests, RequestDetail, ApprovalCenter, UnifiedInbox, WorkflowDesigner, etc.) + frontend/src/services/*.service.ts
- RequestStatus **enum has ~110 statuses** spanning generic, HR/recruitment, IT procurement, Finance, onboarding, offboarding, and inter-company chargeback.

2. Architecture at a glance

Pattern: modular monolith, layered routes → controllers → services → Prisma. All mounted in backend/src/routes/index.ts under /api/v1.

```
HTTP → backend/src/routes/index.ts → sub-routers (per domain)
→ controllers (thin) → services (business logic) → Prisma (PostgreSQL)
```

Key domains & where they live

Domain	Routes	Controllers	Services
Request lifecycle	`request.routes.ts`	`request.controller.ts`	`requestTransition.service.ts`, `availableTransitions.service.ts`, `requestAccess.service.ts`
Service desks / categories / request types	`serviceDesk.routes.ts`	`serviceDesk.controller.ts`	`serviceDesk.service.ts`
Workflow-designer engine	`workflow.routes.ts`, `workflowVersion.routes.ts`, `workflowTransition.routes.ts`, `requestStatusDefinition.routes.ts`	`workflow.controller.ts`, `workflowVersion.controller.ts`	`workflowCompiler.service.ts`, `workflowGraph.service.ts`, `workflowVersion.service.ts`, `workflowBootstrap.service.ts`, `transitionGuards.ts`, `transitionPolicy.service.ts`, `requestStatusDefinition.service.ts`
IT workflow	`it-workflow.routes.ts`	`it-workflow.controller.ts`	auto-assignment, request creation policy
HR workflows	`onboarding/offboarding/loa/interview/screening.routes.ts`	matching controllers	`onboarding.service.ts` + per-flow services
Finance workflows	`finance-workflow.routes.ts`, `chargeback-workflow.routes.ts`	matching controllers	—
ESM travel	`esm-workflow.routes.ts`	`esm-workflow.controller.ts`	—
Approvals	`approval.routes.ts`, `approvalPolicy.routes.ts`, `approvalDelegation.routes.ts`	`approval.controller.ts`	`approvalRuntime.service.ts`, `approvalPolicy.service.ts`, `approvalDelegation.service.ts`
SLA & escalation	`escalationRule.routes.ts`, `sla`	—	`sla.service.ts`, `sla-pause.service.ts`
Notifications	`notification.routes.ts`, `notificationSse.routes.ts`	`notification.controller.ts`	`notification.service.ts`, `outputboxDispatcher.service.ts`
Asset management	`asset.routes.ts`	`asset.controller.ts`	`autoAssignment.service.ts`
Knowledge base	`kb.routes.ts`	`kb.controller.ts`	`kb.service.ts`
Search / reports / insights	`search.routes.ts`, `reports.routes.ts`, `insights.routes.ts`	matching	`insights.service.ts`
Auth / users / RBAC	`auth.routes.ts`, `user.routes.ts`	`auth.controller.ts`, `user.controller.ts`	`permission.service.ts`, `password-reset.service.ts`

3. The two transition systems (read this carefully)

This is the single most important thing to understand in the ESM module — there are **two** sources of truth for request transitions, and the relationship between them confuses newcomers.

- 1 **The published workflow-designer engine (AUTHORITATIVE at runtime).** Workflows are authored in the Workflow Designer UI (frontend/pages/WorkflowDesigner.tsx) and published as versioned graphs (WorkflowType → WorkflowVersion → WorkflowNode + WorkflowEdge). At runtime, requestTransition.service.ts validates and performs transitions **DB-first** against the workflow_transitions table (and published workflow graphs). See 03-request-lifecycle.md and 04-business-flows.md.
- 2 **The hardcoded VALID_TRANSITIONS fallback** (backend/src/utils/workflowTransitions.ts) — a seed/reference map of status → allowed next statuses. **This is NOT the runtime source of truth.** It is consulted only as a fallback when the DB is empty, and as documentation. **Do not edit it expecting runtime behavior to change** — the DB workflow_transitions table and published workflows drive the runtime. (There's also workflow-designer-production-rollouts and workflow-versioning-remediation guidance in the repo's skills.)

> **Rule of thumb:** to change what transitions are allowed at runtime, edit the published workflow graph (or the workflow_transitions rows), **not** VALID_TRANSITIONS.

4. Key cross-cutting controls

Control	Where	What it does
Auth	`middleware/auth.middleware.ts`	JWT (passport-jwt) + refresh; loads roles + permissions
RBAC	`requirePermission(...)`	Fine-grained `request:*`, `admin:*`, `asset:*`, `kb:*`, `report:*`, etc. (loaded in auth, cached in Redis)
Resource authorization	`authorizeResource` + `loadRequestScopeFromParam`	Row-level access to a request/entity; `requestAccess.service.ts` does full policy evaluation (P02-09)
Rate limiting	`apiLimiter` (global)	Applied after auth routes; auth has its own stricter limiter
Validation	`validate(...)` (Joi)	Request body/query validation
File upload	`upload.middleware.ts`	multer + S3 presigned (with AV scan for some flows)
Notifications	`notification.service.ts` + SSE	In-app (SSE, Redis pub/sub) + email; durable via outbox

5. Working in this module — practical rules

- 1 **Runtime transitions are DB/published-workflow driven**, not the hardcoded map. Change the workflow graph, not VALID_TRANSITIONS.
- 2 **Use** requestTransition.service.ts for status changes — it centralizes validation, terminal-status timestamps, SLA pause/resume, auto-assignment, activity + audit logging, notifications, and guard conditions. Do not scatter prisma.request.update({ status }) across controllers.
- 3 **Preserve the layering:** routes → controllers → services → Prisma; controllers stay thin.

- 4 **Respect row-level access.** Request routes that expose a request by ID use `authorizeResource + requestAccess.service`; don't bypass it.
- 5 **SLA pause/resume** is automatic on transitions with `slaPause` workflow steps — be careful when adding new transitions.
- 6 **Notifications are durable** via the outbox pattern (`outboxDispatcher.service.ts`); failures retry with backoff.
- 7 **Tests live beside code** (`__tests__`/ dirs). Run `cd backend && npm test` before/after changes.
- 8 **Soft deletes** via `deletedAt` on many entities (e.g. `Request`); query with `deletedAt: null`.
- 9 **Optimistic concurrency:** `Request.version` increments on writes; transactional workflow commands use version guards.
- 10 **Feature flags** gate some UI (e.g. KB via `isFeatureEnabled('kb')`).

6. Document index (this ESM handover set)

Doc	Covers
00-README.md (this)	Landing page, architecture, the two-transition-system gotcha, controls, working rules
01-codebase-map.md	Directory-by-directory backend + frontend map
02-data-model.md	ESM Prisma models grouped by domain, status enums, invariants
03-request-lifecycle.md	Request lifecycle, status machine, workflow-designer runtime
04-business-flows.md	End-to-end flows: service desks, IT/HR/Finance/ESM-travel, approvals, SLA
05-api-surface.md	ESM API surface: routes, controllers, permissions
06-frontend-map.md	Frontend routes, pages, request UI, workflow designer, services

Pre-existing docs worth reading (not regenerated here)

- `docs/ESM_AUDIT.md` — full stack/domain audit of the whole platform
- `docs/ESM_ARCHITECTURE_PRODUCTION_READINESS_AUDIT.md` — production-readiness audit
- `docs/runbooks/production-deployment.md` + `docs/runbooks/workflow-bootstrap-*.md` — deployment & workflow seeding
- End-user guides in `docs/` (`crm-module-end-user-guide.md`, `new-hiring-request-workflow.md`, `purchase-requisition-workflow.md`, etc.)

7. Quick start (dev)

```
# Backend
cd backend
npm install
cp .env.example .env # set DATABASE_URL, JWT secrets
npm run prisma:generate
npx prisma migrate dev
npm run prisma:seed # seeds service desks, categories, request types, workflow types/transitions
npm run dev # port 3000

# Frontend
cd ../frontend
npm install
npm run dev # port 5173

# Tests
cd backend && npm test
```

Seeded accounts (passwords in .env): admin@test.local, hr@test.local, it@test.local, ceo@test.local, user@helpdesk.com.